ORVL-023

.AXM

Axiom eXchange Model · Modular Execution-Graph Container · Hybrid Trust Model · Successor to GGUF

> *"A GPU asks: how fast can we multiply matrices? .AXM asks: which skill, which proof, which trajectory — and only those."*

Concept Note · Antonio Roberts · Orivael · May 2026

Status: CONCEPT — Software emulator shipped	**Pairs with:** ORVL-004 (MKB) · ORVL-018 (ANF) · ORVL-019 (Mobile)	**Trust model:** Hybrid (open + signed)

Abstract.

The Axiom eXchange Model (.AXM) is a software-emulated successor to GGUF-style model containers. Where GGUF is a flat file of quantised weights, .AXM treats a model as a *living execution graph*: an always-resident Core Logic Module + lazy-loaded Skill Delegates + pre-compiled Trajectory Blocks + a Vector-Vertex DB + a Proof Ledger + a Hardware Map. Each sub-module is HMAC-signed independently under a hybrid trust model (open container, signed delegates). The container drives the existing AXIOM stack on every operation: skill delegates land in the MKB BlockRegistry as `block_type=AXM_SKILL`; every proof entry is verified through the ANF governance coprocessor; the mobile NeuralComputeBlock loads delegates lazily as the active task's intent class changes. The promise is energy-proportional intelligence: the file does not awaken the whole cathedral — it lights only the rooms needed for the current problem.

01 | BEYOND STATIC TENSORS

1. The .AXM Architecture: Beyond Static Tensors

Current formats like GGUF are essentially flat files containing headers and quantised weights. A self-designed AI format would treat the model as a living graph — partitioned, hardware-aware, and able to load only the logic needed for the current task.

Bit-Depth Elasticity (Dynamic Quantisation)

Critical reasoning layers stay at 16-bit BF16 for precision; stylistic / knowledge layers drop to 1.5-bit or 2-bit. A 70B-class model fits into 24 GB VRAM while preserving most of the intelligence of its higher-precision form — the north-star promise of elastic storage.

Integrated Trajectory Blocks

Instead of storing only weights, .AXM contains a HISTORY segment of pre-compiled trajectories — verified motion paths, reasoning traces, or tool-use routes the system can reuse instantly. The format stores not only what the model knows, but also proven pathways for acting on that knowledge.

Design target

An AI-designed model file: the cathedral does not need to wake all at once. Only the rooms relevant to the current intent class light up; the rest stay cold on disk.

02 | PARTITIONED MODULAR STORAGE

2. Six Sub-Modules + Container Layout

Module	Purpose	Runtime behavior
Core Logic Module (The Axiom)	Tiny ultra-fast 1B–3B reasoning core	Always resident — handles routing, safety, verification, intent
Skill Delegates	Task-specific plugin files, each HMAC-signed	Loaded into VRAM only when a WHEN-condition matches
Trajectory Blocks (HISTORY)	Verified solution paths, action patterns, tool chains	Pulled when a task matches prior proven behavior
Vector-Vertex DB	Geometry primitive ↔ semantic class map	Vision-to-pattern acceleration; format-level class lookup
Proof Ledger	Per-module HMAC signatures + content hashes	Verified before any skill activates at runtime
Hardware Map	Chip-aware dispatch (cpu / gpu / npu / fpga / compile_on_load)	Chosen at load time, drives the ANF dispatch path

Container layout on disk:

```
my_model.axm/
■■■ header.json ← Semantic State Header (signed)
■■■ core/core.json ← Core Logic Module manifest
■■■ delegates/ ← lazy-loaded on WHEN match
■ ■■■ pii_redactor/skill.json
■ ■■■ anf_governance/skill.json
■ ■■■ vector_recall/skill.json
■■■ trajectories/ ← pre-compiled reasoning paths
■ ■■■ history.jsonl
■■■ vertices.json ← Vector-Vertex DB
■■■ proofs/ledger.jsonl ← per-module HMAC + sha256
```

3. The State-Space Header

GGUF has a practical key-value header. .AXM goes further: a Semantic State Header that describes execution paths, hardware dispatch, proof conditions, and vertex/vector indexes — the metadata is itself an executable contract.

Feature	.GGUF	.AXM
Indexing	Metadata strings (name, version, arch).	Vertex-mapped indexes tied to spatial / semantic / task-state clusters.
Quantization	Uniform schemes such as Q4_K_M.	Non-linear, per-layer elasticity driven by task importance.
Execution	CPU/GPU offloading decided by runtime.	Direct hardware mapping with chip-aware JIT execution paths.
Verification	Simple file hashes and metadata checks.	Per-module HMAC signatures + ANF-coprocessor proof verification.

Header sketch (as serialised in header.json):

```
{
"format_version" : "0.1-concept",
"core_logic" : "axiom_core_3b",
"quant_map" : "elastic_per_layer",
"hardware_map" : "compile_on_load",
"delegates" : ["pii_redactor", "anf_governance", "vector_recall"],
"safety_proofs" : true,
"signature" : "<HMAC-SHA256 over canonical payload>"
}
```

4. Wiring Into Three Existing Patents

.AXM is not a stand-alone artefact. The container exists to exercise three patents already in the AXIOM portfolio. Every operation drives them.

Patent	Role for AXM
ORVL-004 MKB	Each loaded Skill Delegate registers as a `KnowledgeBlock` with `block_type="AXM_SKILL"` in the existing `BlockRegistry`. AXM is the on-disk container; MKB is the live registry.
ORVL-018 ANF	`verify_proofs()` drives `GovernanceCoprocessorEmulator.process()` once per proof entry. The header's `hardware_map` selects the ANF dispatch class.
ORVL-019 Mobile	`NeuralComputeBlock(axm_container=…)`; `pre_classify()` lazy-loads delegates whose WHEN-conditions match the classifier's intent class — the same power model the phone already uses.

> **Energy-proportional intelligence**
> An **INFORM** task lights the smooth-convergence pathway (20 ANF cores active, two skill delegates loaded). A **HARM** task lights only the boundary-detection fast-path (5 ANF cores, one delegate). Safe inference uses more compute than dangerous inference detected — the inverse of every other AI system.

5. Hybrid Trust Model — Open Container, Signed Delegates

The AXM brief's strategic fork (proprietary / open / hybrid) is answered with hybrid. Three derived keys handle three independent signing surfaces — every artefact is verifiable; no artefact is encrypted; ecosystem partners can ship signed delegates without key exchange.

Layer	Key	What it signs
Container	derive_key(b'axiom-axm-container-v1')	header.json + AXMRouteResult
Delegates	derive_key(b'axiom-axm-delegate-v1')	each delegate's skill.json
Proofs / vectors / trajectories	derive_key(b'axiom-axm-proof-v1')	ledger.jsonl + vertices.json + history.jsonl

Verification flow (sample run):

```
$ python -m axiom_axm verify /tmp/starter.axm
{
"verified": true,
"proofs_checked": 6,
"fingerprint": "f5481d89"
}

$ python -m axiom_axm route /tmp/starter.axm "explain transformers"
{
"task": "explain transformers",
"intent_class": "INFORM", "confidence": 0.55,
"loaded_skills": ["anf_governance", "pii_redactor"],
"skipped_skills": ["vector_recall"],
"anf_distance": 0.000, "anf_cores_active": 20,
"signature": "<64-char HMAC>"
}
```

6. Provisional Patent Claims (ORVL-023)

Claim 1. A model container architecture wherein a model is partitioned into an always-resident Core Logic Module and a set of independently HMAC-signed Skill Delegates, with each delegate gated by a WHEN-condition over a classifier-emitted intent class, such that VRAM residency is bounded by the union of delegates whose WHEN-conditions match the current task — yielding compute and memory proportional to task complexity rather than model size.

Claim 2. A Semantic State Header that replaces flat metadata with an executable contract describing per-layer quantisation elasticity, hardware-dispatch class, declared delegate set, and a safety-proof requirement flag, wherein the header is HMAC-signed under a container-scoped key and load-time signature failure refuses any subsequent module access.

Claim 3. A Proof Ledger embedded in the container — one HMAC-signed entry per sub-module bound to a content SHA-256 of that sub-module's on-disk file — verified via a governance-coprocessor pipeline before any skill delegate may be activated, providing cryptographic binding between disk integrity and runtime permission.

Claim 4. Lazy WHEN-triggered skill activation in which a classifier emits an intent class per task, the container compares the intent class against per-delegate WHEN conditions, and matched delegates are registered with an external Modular Knowledge Block registry as first-class blocks of type AXM_SKILL — giving the runtime uniform inspection of statically-shipped and dynamically-composed skills.

Claim 5. A Vector-Vertex Database embedded at the container level mapping perception classes (e.g. CYLINDRICAL, FRAGILE, PLANAR) to vertex primitive clusters, signed under the proof key and consulted before geometric inference — collapsing the perception-to-skill lookup into a format-level table read.

Claim 6. A hybrid trust model in which the container header and per-skill delegate manifests are signed under distinct derived keys, and the runtime refuses to load any artefact whose signature does not verify under the corresponding key — enabling third-party delegate publication without key exchange while preserving load-time integrity guarantees.

07 | PATENT PORTFOLIO RELATIONSHIP

7. Patent Portfolio Relationship

Patent	Role in ORVL-023
ORVL-001 Constitutional Language	`.axiom` spec at `axiom_files/core/axiom_axm.axiom` is the constitutional contract for the container.
ORVL-004 MKB	Each loaded delegate registers as a KnowledgeBlock — the live registry uniformly inspects shipped + composed skills.
ORVL-005 Latent / MonotonicGate	AXM Trajectory Blocks store proven reasoning trajectories that the gate can short-circuit to without re-derivation.
ORVL-010 CANNOT_MUTATE	Container header, format_version, and proof_ledger are declared CANNOT_MUTATE in the .axiom spec — runtime enforces frozen-dataclass discipline.
ORVL-014 Constitutional World Model	Vector-Vertex DB provides the geometry primitives that the world model uses for perception-to-skill lookup.
ORVL-016 Intent Typing	The classifier whose intent class drives WHEN matching.
ORVL-018 ANF	Governance Coprocessor verifies the Proof Ledger; Hardware Map selects ANF dispatch path.
ORVL-019 Mobile / ASPA	Phone's NeuralComputeBlock accepts an AXM container and lazy-loads delegates per intent class on-device.
ORVL-022 CPI	Physical skill delegates (Wrap-Grip, Pinch-Pressure, etc.) ship as AXM Skill Delegates; vertex classes match the AXM Vector-Vertex DB.

> *"Twenty-three patents. One constitutional AI architecture. From language reasoning to physical motion to model containers — the same geometry, lit one room at a time."*

CONFIDENTIAL CONCEPT NOTE — ORVL-023 · Orivael · Antonio Roberts · hello@orivael.dev · github.com/Orivael-Dev/axiom · May 2026